

From the Weak Formulation to Working Code: Implementation in FEniCSx, and the Computational Results

M. Javed Bukhari

August 11, 2026

Contents

I	How FEniCSx works	2
1	The idea	2
2	Meshes	3
2.1	Mathematics	3
2.2	Code	3
3	Function spaces	3
3.1	Mathematics	3
3.2	Code	3
4	Forms	4
4.1	Mathematics	4
4.2	Code	4
4.3	Operators not built in	5
5	Boundary conditions	5
5.1	Mathematics	5
5.2	Code	5
5.3	Conditions on a subspace	6
6	Solving	6
7	Mixed spaces	7
7.1	Mathematics	7
7.2	Code	7
8	Manufactured solutions	7
8.1	Mathematics	7
8.2	Code	8
9	Measuring the error	8
9.1	Mathematics	8
9.2	Code	9
10	Tests	9

II	Building the three solvers	10
11	Why three stages	10
12	Stage 1: Poisson	10
12.1	Mathematics	10
12.2	Code	10
13	Stage 2: Stokes	11
13.1	Mathematics	11
13.2	Code	11
13.3	The pressure nullspace	11
14	Stage 3: micropolar	12
14.1	Mathematics	12
14.2	Code	12
14.3	A check on the coupling	13
III	Results	13
15	Convergence	13
15.1	Poisson	13
15.2	Stokes	13
15.3	Micropolar	14
16	The parameter study	14
16.1	What it tests	14
16.2	Rates across six decades	14
16.3	Growth of the error	15
16.4	Interpretation	15
17	Summary	16

Structure. Part I explains how FEniCSx works, taking each construct in turn and pairing the mathematical object with the line of code that expresses it. Part II builds the three solvers in sequence, Poisson then Stokes then micropolar, with the reasoning behind each design decision. Part III reports the computational results and what they establish. All code is for DOLFINx version 0.11, whose interface differs from earlier versions in several places that are noted where they arise.

Part I

How FEniCSx works

1 The idea

The weak formulation of a boundary value problem has the shape: find u in some space V such that

$$a(u, v) = F(v) \quad \text{for all } v \in V.$$

Discretising replaces V by a finite dimensional subspace V_h , which turns the statement into a linear system. Traditionally one then writes code to build that system: loop over elements,

evaluate basis functions at quadrature points, compute local matrices, assemble them into a global matrix, apply boundary conditions, call a solver.

FEniCSx removes that entire layer. You write a and F in a notation close to the mathematics, declare which space V_h is, and the library generates and compiles the assembly code itself. What you write is the formulation. What the computer does with it is not your concern until something goes wrong.

This means the correctness of the code is essentially the correctness of the weak formulation. A sign error in the form is a sign error in the mathematics, not a bug in a loop. That is a considerable advantage, and it is why an error in a FEniCSx program is usually found by rereading the derivation rather than by debugging.

The system is built from four components. UFL is the language in which forms are written. Basix supplies the finite elements, that is, the basis functions and the quadrature rules. FFCx compiles a form into C code. DOLFINx holds the mesh, the function spaces and the solution vectors, and talks to PETSc for the linear algebra.

2 Meshes

2.1 Mathematics

The domain Ω is partitioned into triangles K meeting edge to edge. Writing h_K for the diameter of K and ρ_K for the diameter of its inscribed circle, the family of meshes must be shape regular,

$$\frac{h_K}{\rho_K} \leq \sigma \quad \forall K \in \mathcal{T}_h, \forall h > 0,$$

so that the triangles do not degenerate as $h \rightarrow 0$. Uniform triangulations of the unit square satisfy this automatically.

2.2 Code

```
from dolfinx import mesh as dmesh
from mpi4py import MPI

msh = dmesh.create_unit_square(MPI.COMM_WORLD, n, n)
```

This produces $2n^2$ triangles, obtained by splitting each cell of an $n \times n$ grid of squares along a diagonal, so $h = 1/n$.

The communicator argument is present because DOLFINx is parallel by default. Running the same script under `mpirun` distributes the mesh across processes without any change to the code. In serial, `MPI.COMM_WORLD` simply denotes a single process. The consequence to remember is that any quantity computed by integration is local to a process and must be summed across processes before it means anything; this is why the error routines in Section 9 contain an `allreduce`.

Useful properties of the mesh object:

```
msh.topology.dim # 2 for a plane mesh
msh.topology.dim - 1 # 1, the dimension of the facets (edges)
msh.geometry.dim # 2, the dimension of the ambient space
msh.basix_cell() # the reference cell, needed to build elements
```

The distinction between topological and geometric dimension matters for surfaces embedded in three dimensions, where they differ. Here they agree, but the code should use whichever is meant.

3 Function spaces

3.1 Mathematics

The discrete space is

$$\mathcal{L}_k = \{ \phi \in C(\overline{\Omega}) : \phi|_K \in \mathbb{P}_k(K) \ \forall K \in \mathcal{T}_h \},$$

continuous functions that are polynomial of degree k on each triangle. Continuity is what makes $\mathcal{L}_k \subset H^1(\Omega)$: a piecewise polynomial with a jump across an edge has a weak derivative containing a Dirac measure supported on that edge, which is not an $L^2(\Omega)$ function. So continuity is exactly the condition for conformity, not a matter of taste.

3.2 Code

```
from dolfinx import fem

V = fem.functionspace(msh, ("Lagrange", 2))
```

The name **Lagrange** refers to the standard nodal basis, in which each basis function equals one at its own node and zero at every other. Degree 2 on a triangle means six degrees of freedom, three at the vertices and three at the edge midpoints.

For a vector valued space, the shape must be given:

```
V = fem.functionspace(msh, ("Lagrange", 2, (msh.geometry.dim,)))
```

Remark 3.1 (Version difference). In DOLFINx 0.7 and earlier this function was called **FunctionSpace** with a capital F, and vector spaces were built with a separate **VectorFunctionSpace**. Both were replaced at 0.8. A large amount of material online predates that change, so a tutorial that does not match the installed version is usually the tutorial that is out of date.

For finer control, and necessarily for mixed spaces, build the element explicitly through **Basix**:

```
import basix.ufl

cell = msh.basix_cell()
element = basix.ufl.element("Lagrange", cell, 2, shape=(msh.geometry.dim,))
V = fem.functionspace(msh, element)
```

4 Forms

4.1 Mathematics

For the Poisson problem the weak formulation is: find $u \in H_0^1(\Omega)$ with

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx = F(v) \quad \forall v \in H_0^1(\Omega).$$

4.2 Code

```
import ufl

u = ufl.TrialFunction(V)
v = ufl.TestFunction(V)

a = ufl.inner(ufl.grad(u), ufl.grad(v)) * ufl.dx
L = ufl.inner(f, v) * ufl.dx
```

The correspondence is direct. A trial function is the unknown, appearing in the bilinear form; a test function is the arbitrary function the equation is tested against. Neither is a numerical object: they are symbols, and the form is a symbolic expression which FFCx later compiles.

The measure `ufl.dx` denotes integration over the domain and `ufl.ds` integration over its boundary. Multiplying an expression by a measure is what turns it into a form.

Operators available include `grad`, `div`, `curl`, `nabla_grad`, `inner`, `dot`, `outer`, `sym`, `tr`, and elementary functions such as `sin`, `cos`, `exp`, `sqrt`. Spatial derivatives of a single component are written `u[1].dx(0)`, meaning $\partial_1 u_2$.

Remark 4.1 (inner against dot). For real valued problems these agree. `inner` conjugates its second argument and is the right choice in general; `dot` contracts only the last index of the first argument with the first index of the second. Using `inner` everywhere is a good habit.

4.3 Operators not built in

The two dimensional operators of the micropolar system have no direct UFL equivalent and must be defined. With $\nabla^\perp w = (\partial_2 w, -\partial_1 w)$ and $\text{curl } \mathbf{v} = \partial_1 v_2 - \partial_2 v_1$:

```
def grad_perp(w):
    """Perpendicular gradient, the gradient rotated by ninety degrees."""
    g = ufl.grad(w)
    return ufl.as_vector([g[1], -g[0]])

def curl_2d(u):
    """Scalar curl of a plane vector field."""
    return u[1].dx(0) - u[0].dx(1)
```

The reason these need defining is that UFL's `curl` expects a three dimensional vector and returns a vector. In two dimensions only the out of plane component survives, so it is treated as a scalar, and $\nabla^\perp$ maps in the opposite direction, from scalars to vectors. These two operators are how the micropolar system converts between the vector velocity and the scalar microrotation, and by the rotation identity they are formal adjoints on $H_0^1(\Omega)$.

5 Boundary conditions

5.1 Mathematics

The condition $u = 0$ on $\partial\Omega$ is imposed by working in $H_0^1(\Omega)$ rather than $H^1(\Omega)$. Discretely this means constraining the degrees of freedom that lie on the boundary. Recall from the derivations that it is the vanishing of the *test* function that removes the boundary term in the integration by parts, so the constraint is what makes the weak formulation the one that was derived.

5.2 Code

Three steps: find the boundary facets, find the degrees of freedom on them, build the condition.

```
import numpy as np
from dolfinx import mesh as dmesh, fem
from petsc4py import PETSc

fdim = msh.topology.dim - 1

facets = dmesh.locate_entities_boundary(
    msh, fdim, lambda x: np.full(x.shape[1], True)
)
dofs = fem.locate_dofs_topological(V, fdim, facets)
```

```
bc = fem.dirichletbc(PETSc.ScalarType(0.0), dofs, V)
```

The marker function receives coordinates as an array of shape `(gdim, npoints)` and must return a boolean array of length `npoints`. Returning `True` everywhere selects the whole boundary, which is safe because `locate_entities_boundary` only ever offers boundary entities as candidates in the first place.

Selecting part of the boundary is done by returning a condition instead:

```
lambda x: np.isclose(x[0], 0.0) # left edge
lambda x: np.isclose(x[0], 0.0) | np.isclose(x[0], 1.0) # left and right
```

Note the bitwise operators `&` and `|` rather than `and` and `or`, because these act elementwise on arrays.

5.3 Conditions on a subspace

In a mixed space the condition applies to one field only, and this requires a different call. The degrees of freedom must be identified in the numbering of the mixed space and in the numbering of the standalone subspace at the same time, so both are passed:

```
V, _ = W.sub(0).collapse()

facets = dmesh.locate_entities_boundary(
    msh, fdim, lambda x: np.full(x.shape[1], True)
)
dofs = fem.locate_dofs_topological((W.sub(0), V), fdim, facets)

zero = fem.Function(V)
zero.x.array[:] = 0.0

bc = fem.dirichletbc(zero, dofs, W.sub(0))
```

Here `W.sub(0)` is a view of the velocity component inside the mixed space, and `collapse()` produces a standalone space with its own numbering together with a list mapping its degrees of freedom into the mixed array. Passing the pair `(W.sub(0), V)` is required; passing only `W.sub(0)` fails, and the resulting error message is not informative.

The boundary value is supplied as a `Function` on the collapsed space rather than as a constant, because the condition is vector valued.

6 Solving

```
from dolfinx.fem.petsc import LinearProblem

DIRECT_SOLVER = {
    "ksp_type": "preonly",
    "pc_type": "lu",
    "pc_factor_mat_solver_type": "mumps",
}

problem = LinearProblem(
    a, L, bcs=[bc],
    petsc_options=DIRECT_SOLVER,
    petsc_options_prefix="poisson_",
)
uh = problem.solve()
```

The options select a direct solver. Setting `ksp_type` to `preonly` means no Krylov iteration is performed, and `pc_type` set to `lu` means the preconditioner is a complete LU factorisation, so the combination is an exact solve. At the mesh sizes used here this is faster than any iterative method and, more importantly for a convergence study, it removes solver tolerance as a possible source of error. An iterative solver stopped at a loose tolerance can contaminate a convergence rate in a way that is tedious to diagnose.

MUMPS is a sparse direct solver that handles indefinite systems, which matters from Stokes onwards.

Remark 6.1 (Version difference). DOLFINx 0.11 made `petsc_options_prefix` a required keyword argument. PETSc maintains one global options database, so when a program solves several different systems the prefix namespaces each solver's settings and prevents collisions. Any unique string works; the convention is lowercase with a trailing underscore. Earlier versions did not require it, and omitting it in 0.11 raises a `TypeError` whose message is preceded by an unrelated `AttributeError` from the destructor firing on a partly built object. The lesson is to read the bottom of a traceback first.

7 Mixed spaces

7.1 Mathematics

The micropolar problem has three unknowns living in three different spaces,

$$\mathbf{V}_h = [\mathcal{L}_2^0]^2, \quad Q_h = \mathcal{L}_1 \cap L_0^2(\Omega), \quad W_h = \mathcal{L}_2^0.$$

As in the analysis, they are treated as one unknown in the product space, which is what allows a single bilinear form and a single solve.

7.2 Code

```
import basix.ufl

cell = msh.basix_cell()

velocity = basix.ufl.element("Lagrange", cell, degree, shape=(msh.geometry.dim,))
pressure = basix.ufl.element("Lagrange", cell, degree - 1)
rotation = basix.ufl.element("Lagrange", cell, degree)

W = fem.functionspace(
    msh, basix.ufl.mixed_element([velocity, pressure, rotation])
)
```

The velocity degree exceeds the pressure degree by one. This is the Taylor–Hood choice, and it is what supplies the discrete inf-sup condition; the equal order pair fails, and fails by producing a spurious pressure mode rather than by failing to solve. The microrotation is unconstrained by inf-sup considerations, since it does not appear in the constraint form, and is given the same degree as the velocity so that it converges at the same rate.

Trial and test functions come as tuples:

```
u, p, w = ufl.TrialFunctions(W)
v, q, z = ufl.TestFunctions(W)
```

Note the plural. The singular forms `TrialFunction` and `TestFunction` return a single object on the whole mixed space, which is not what is wanted here.

To extract components from a computed solution:

```
uh, ph, wh = ufl.split(solution)
```

This yields UFL expressions suitable for use in forms, which is what the error computation needs. A separate operation, `solution.split()`, produces `Function` objects instead, which is what is wanted for output to file.

8 Manufactured solutions

8.1 Mathematics

To verify a discretisation, choose exact fields satisfying the boundary conditions and any constraints, substitute them into the strong form to obtain the data they solve, run the code with that data, and compare against the known answer. If the observed convergence rates match the theoretical ones the discretisation is correct.

The test is sharp because the rates are integers predicted in advance. A rate of 2.4 where the theory says 3 is not a partial success but a defect.

8.2 Code

The essential technique is to define the exact solution as a UFL expression and obtain the forcing by symbolic differentiation, in the same language:

```
def poisson_exact(mesh):
    x = ufl.SpatialCoordinate(mesh)
    return ufl.sin(ufl.pi * x[0]) * ufl.sin(ufl.pi * x[1])

def poisson_source(mesh):
    return -ufl.div(ufl.grad(poisson_exact(mesh)))
```

There is no transcription step. Deriving f by hand, or with a separate symbolic package, and then typing it into the code introduces an opportunity for a sign error which is invisible until the rates come out wrong. Doing the differentiation in UFL means the forcing is consistent with the operator actually being discretised by construction. For the micropolar sources, which are lengthy by hand, this is not a convenience but a necessity.

The exact velocity is built from a stream function so that it is divergence free identically:

```
def velocity_exact(mesh):
    x = ufl.SpatialCoordinate(mesh)
    psi = ufl.sin(ufl.pi * x[0])**2 * ufl.sin(ufl.pi * x[1])**2
    return grad_perp(psi)
```

Since $\operatorname{div} \nabla^\perp \psi = \partial_1 \partial_2 \psi - \partial_2 \partial_1 \psi = 0$, the incompressibility constraint is satisfied exactly rather than approximately, so the manufactured problem is a genuine instance of the system and not a perturbation of one. Because ψ and its gradient both vanish on $\partial\Omega$, the velocity satisfies the no slip condition.

The pressure is chosen with zero mean,

$$p = \cos(\pi x) \cos(\pi y), \quad \int_{\Omega} p \, dx = \left(\int_0^1 \cos(\pi s) \, ds \right)^2 = 0,$$

so that it lies in $L_0^2(\Omega)$ and is the unique representative of its equivalence class.

9 Measuring the error

9.1 Mathematics

The quantities of interest are

$$\|u - u_h\|_{L^2} \quad \text{and} \quad \|\nabla(u - u_h)\|_{L^2},$$

and the observed rate between successive meshes is

$$r_m = \frac{\log(e_{m-1}/e_m)}{\log(h_{m-1}/h_m)}.$$

9.2 Code

```
def error_L2(uh, u_exact, mesh, degree):
    e = uh - u_exact
    dx = ufl.dx(metadata={"quadrature_degree": 2 * degree + 3})
    local = fem.assemble_scalar(fem.form(ufl.inner(e, e) * dx))
    return np.sqrt(mesh.comm.allreduce(local, op=MPI.SUM))
```

Three points in this short function matter.

The exact solution is never interpolated into a finite element space. It enters as a UFL expression and is evaluated at quadrature points directly. Interpolating it into the same space used for the solution and then differencing nodal values measures the interpolation error rather than the discretisation error, and the observed rate plateaus for reasons unrelated to the scheme. This is the single most common way to obtain wrong convergence rates, and avoiding it entirely is better than compensating for it.

The quadrature degree is raised to $2k + 3$. The integrand is a product of two functions each of degree k against an exact solution which is not polynomial at all, so default quadrature is not sufficient and the measured error is polluted by quadrature error.

The result is summed across processes. `assemble_scalar` returns the contribution from the local part of the mesh only, so without the `allreduce` a parallel run silently reports the error on a fraction of the domain.

10 Tests

Convergence rates should be asserted, not plotted. A figure that a person must inspect is not a test, and it does not run in continuous integration.

```
def test_L2_convergence_rate():
    errors = [compute_error(n) for n in [8, 16, 32, 64]]
    sizes = [1.0 / n for n in [8, 16, 32, 64]]
    rates = convergence_rates(sizes, errors)
    assert rates[-1] > 2.9, (
        f"L2 rate is {rates[-1]:.3f}, expected 3. "
        f"All rates: {np.array2string(rates, precision=3)}"
    )
```

The tolerance of 0.1 is tight enough to catch a genuine defect and loose enough to absorb the small preasymptotic deviation on the coarsest pair. Printing the whole sequence in the failure message is worth the extra line, because the shape of the sequence distinguishes a preasymptotic transient from a real defect: a transient descends steadily toward the theoretical value, whereas a defect sits flat at the wrong number.

Some care is needed about what a test asserts. An early version of the boundary condition test in this project inspected particular entries of the solution array, which required composing

two different degree of freedom numberings and broke twice for reasons having nothing to do with the solver. Replacing it by a surface integral,

```
ds = ufl.ds(metadata={"quadrature_degree": 6})
local = fem.assemble_scalar(fem.form(ufl.inner(uh, uh) * ds))
norm = np.sqrt(msh.comm.allreduce(local, op=MPI.SUM))
assert norm < 1e-12
```

fixed it immediately. The second version asserts the mathematical property, that the trace vanishes on $\partial\Omega$, rather than a detail of how the mixed space happens to be numbered, and it therefore remains valid if the numbering changes. Tests that assert mathematics are more robust than tests that assert implementation details.

Part II

Building the three solvers

11 Why three stages

The solvers were built in sequence: Poisson, then Stokes, then micropolar. Each was verified before the next was started.

The reason is diagnostic. Writing the coupled three field solver first and finding a wrong convergence rate leaves four candidate causes at once: the mixed space, the pressure nullspace, the coupling signs, and the manufactured forcing. Isolating one from four is slow. Building in stages means that at each step almost everything is already verified, so a wrong rate has one candidate cause, namely whatever was added.

The infrastructure is written once and reused. The error routines, the rate computation and the test structure are identical at all three stages; only the forms and the spaces change.

12 Stage 1: Poisson

12.1 Mathematics

Find $u \in H_0^1(\Omega)$ with

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx \quad \forall v \in H_0^1(\Omega).$$

12.2 Code

```
def solve(n, degree=2, comm=MPI.COMM_WORLD):
    msh = create_mesh(n, comm)
    V = fem.functionspace(msh, ("Lagrange", degree))

    bc = homogeneous_dirichlet_bc(V, msh)
    f = poisson_source(msh)

    u = ufl.TrialFunction(V)
    v = ufl.TestFunction(V)

    a = ufl.inner(ufl.grad(u), ufl.grad(v)) * ufl.dx
    L = ufl.inner(f, v) * ufl.dx

    problem = LinearProblem(
        a, L, bcs=[bc],
```

```

    petsc_options=DIRECT_SOLVER,
    petsc_options_prefix="poisson_",
)
return problem.solve(), msh

```

The mesh is returned alongside the solution because the caller needs it to build the exact solution as a UFL expression over the same spatial coordinate. Passing a different mesh object would produce a silently meaningless comparison.

Nothing here is difficult. The purpose of the stage is to establish that the toolchain, the manufactured solution and the error measurement are all correct.

13 Stage 2: Stokes

13.1 Mathematics

Find $(\mathbf{u}, p) \in \mathbf{V} \times Q$ with

$$\begin{aligned} \nu (\nabla \mathbf{u}, \nabla \mathbf{v}) - (p, \operatorname{div} \mathbf{v}) &= (\mathbf{f}, \mathbf{v}), \\ -(q, \operatorname{div} \mathbf{u}) &= 0, \end{aligned}$$

for all $(\mathbf{v}, q)$. Three things appear here that had no analogue at Stage 1: the mixed space, the indefiniteness of the resulting matrix, and the constant pressure nullspace.

13.2 Code

```

def bilinear_form(W, nu):
    u, p = ufl.TrialFunctions(W)
    v, q = ufl.TestFunctions(W)

    return (
        nu * ufl.inner(ufl.grad(u), ufl.grad(v)) * ufl.dx
        - ufl.inner(p, ufl.div(v)) * ufl.dx
        - ufl.inner(q, ufl.div(u)) * ufl.dx
    )

```

The two constraint terms carry the same sign, which makes the matrix symmetric. There is no term pairing p with q , which is the algebraic expression of the fact that the pressure never appears against itself, and hence of the failure of coercivity established in Lesson 3.

13.3 The pressure nullspace

The pressure appears only through its gradient, so p and $p + C$ give the same velocity for every constant C . Discretely the vector representing a constant pressure lies in the kernel of the constraint block, the matrix is singular, and the factorisation fails outright. Restricting the pressure space to $L_0^2(\Omega)$ in the analysis and removing this mode in the code are the same act.

```

def pressure_pin(W, msh):
    Q, _ = W.sub(1).collapse()

    def corner(x):
        return np.isclose(x[0], 0.0) & np.isclose(x[1], 0.0)

    dofs = fem.locate_dofs_geometrical((W.sub(1), Q), corner)

    zero = fem.Function(Q)
    zero.x.array[:] = 0.0

```

```
return fem.dirichletbc(zero, dofs, W.sub(1))
```

Note that this uses `locate_dofs_geometrical` rather than `locate_dofs_topological`, because the target is a point rather than a set of facets.

Pinning fixes the additive constant, but not at the value that places the pressure in $L_0^2(\Omega)$. Since the exact pressure was constructed with zero mean, the computed pressure must be shifted before the two are compared, or the error will be dominated by a constant offset and the rate will be meaningless.

```
def normalise_pressure(wh, W, msh):
    _, p_map = W.sub(1).collapse()
    _, ph = ufl.split(wh)

    mean_local = fem.assemble_scalar(fem.form(ph * ufl.dx))
    area_local = fem.assemble_scalar(fem.form(fem.Constant(msh, 1.0) * ufl.dx))

    mean = msh.comm.allreduce(mean_local, op=MPI.SUM)
    area = msh.comm.allreduce(area_local, op=MPI.SUM)

    wh.x.array[p_map] -= mean / area
    wh.x.scatter_forward()
```

The call to `scatter_forward` updates the ghost values held by neighbouring processes after a direct modification of the local array, and is required for correctness in parallel.

An alternative to pinning is to attach a PETSc nullspace object to the matrix, which is cleaner in the sense that it does not impose a pointwise constraint the continuous problem does not have. Pinning was used here because it is simpler, and the observed pressure rate confirms it is harmless for this discretisation.

14 Stage 3: micropolar

14.1 Mathematics

Find $(\mathbf{u}, p, w)$ with

$$(\nu + \nu_r)(\nabla \mathbf{u}, \nabla \mathbf{v}) + c(\nabla w, \nabla z) + 4\nu_r(w, z) - 2\nu_r(\nabla^\perp w, \mathbf{v}) - 2\nu_r(\text{curl } \mathbf{u}, z) - (p, \text{div } \mathbf{v}) - (q, \text{div } \mathbf{u}) = (\mathbf{f}, \mathbf{v}) + (g, z)$$

for all $(\mathbf{v}, q, z)$.

14.2 Code

```
def bilinear_form(W, msh, nu, nu_r, c):
    u, p, w = ufl.TrialFunctions(W)
    v, q, z = ufl.TestFunctions(W)

    nu_ = fem.Constant(msh, float(nu))
    nu_r_ = fem.Constant(msh, float(nu_r))
    c_ = fem.Constant(msh, float(c))

    viscous = (nu_ + nu_r_) * ufl.inner(ufl.grad(u), ufl.grad(v)) * ufl.dx
    angular = c_ * ufl.inner(ufl.grad(w), ufl.grad(z)) * ufl.dx
    damping = 4.0 * nu_r_ * ufl.inner(w, z) * ufl.dx

    coupling_uw = -2.0 * nu_r_ * ufl.inner(grad_perp(w), v) * ufl.dx
```

```

coupling_wu = -2.0 * nu_r_ * ufl.inner(curl_2d(u), z) * ufl.dx

constraint = (
    -ufl.inner(p, ufl.div(v)) * ufl.dx
    - ufl.inner(q, ufl.div(u)) * ufl.dx
)

return viscous + angular + damping + coupling_uw + coupling_wu + constraint

```

The correspondence with the mathematics is line by line, which is the property that makes an error in the code an error in the derivation.

Remark 14.1 (Why the parameters are wrapped as constants). Writing the coefficients as Python floats works until one of them is zero. At $\nu_r = 0$ the expression `4.0 * 0.0 * inner(w, z)` is collapsed by UFL to a symbolic zero, which carries no integration domain, and multiplying it by `ufl.dx` then raises `ValueError: This integral is missing an integration domain`. Wrapping the coefficients as `fem.Constant` objects attaches them to the mesh, so the term retains its domain and the form assembles correctly at any parameter value. This surfaced only when the decoupling limit was tested, which is an argument for testing limits.

14.3 A check on the coupling

The two coupling terms are the most likely place for a sign error, and the rotation identity provides a check independent of the convergence rates. Since $(\nabla^\perp w, \mathbf{v}) = (w, \text{curl } \mathbf{v})$ for fields with zero trace, the two off diagonal blocks must be transposes of one another. This can be verified numerically:

```

lhs = fem.assemble_scalar(fem.form(ufl.inner(grad_perp(w), uu) * ufl.dx))
rhs = fem.assemble_scalar(fem.form(ufl.inner(w, curl_2d(uu)) * ufl.dx))
assert abs(lhs - rhs) < 1e-10 * max(1.0, abs(lhs))

```

If the velocity rates are correct and the microrotation rate is not, this test localises the fault to one of the two coupling terms.

Part III

Results

15 Convergence

All computations are on the unit square with uniform triangulations, using $\mathbb{P}_2$ for velocity and microrotation and $\mathbb{P}_1$ for pressure.

15.1 Poisson

h	L^2 error	rate	H^1 error	rate
1/8	5.481×10^{-4}		3.339×10^{-2}	
1/16	6.874×10^{-5}	3.00	8.419×10^{-3}	1.99
1/32	8.601×10^{-6}	3.00	2.110×10^{-3}	2.00
1/64	1.075×10^{-6}	3.00	5.277×10^{-4}	2.00
1/128	1.344×10^{-7}	3.00	1.319×10^{-4}	2.00

Theoretical rates 3 and 2, matched to two decimal places across four refinements, with the error spanning four orders of magnitude and no sign of a floor.

15.2 Stokes

h	$\mathbf{u}, L^2$	rate	$\mathbf{u}, H^1$	rate	p, L^2	rate
1/8	1.052×10^{-2}		6.166×10^{-1}		2.835×10^{-2}	
1/16	1.331×10^{-3}	2.98	1.587×10^{-1}	1.96	2.745×10^{-3}	3.37
1/32	1.672×10^{-4}	2.99	4.000×10^{-2}	1.99	4.423×10^{-4}	2.63
1/64	2.093×10^{-5}	3.00	1.002×10^{-2}	2.00	1.017×10^{-4}	2.12

The pressure column approaches 2 from above, running 3.37, 2.63, 2.12. This pattern is benign and is worth recognising. The pressure error carries a contribution from the velocity error, which converges at $O(h^3)$ and therefore dominates on coarse meshes before the genuine $O(h^2)$ pressure term takes over. The signature of an unstable element pair is the opposite: a pressure rate that degrades under refinement, or fails to settle, while the velocity rates remain clean.

15.3 Micropolar

At $\nu = \nu_r = c = 1$:

h	$\mathbf{u}, L^2$	$\mathbf{u}, H^1$	p, L^2	w, L^2	w, H^1
1/16	2.99	1.96	3.55	3.64	2.07
1/32	2.99	1.99	3.11	3.41	2.02
1/64	3.00	2.00	2.37	3.16	2.00
1/128	3.00	2.00	2.06	3.05	2.00

Rates as predicted, with the pressure and microrotation L^2 columns again approaching their asymptotic values from above.

16 The parameter study

16.1 What it tests

The convergence tables verify the implementation. They cannot verify the analysis, because a misconception shared between the derivation and the code would pass unnoticed.

The parameter study is different. The analysis predicts an error constant proportional to M/α , where the boundedness constant $M = \nu + c + 9\nu_r$ grows linearly in the vortex viscosity while the coercivity constant $\alpha = \min(\nu, c)/(C_\Omega^2 + 1)$ does not. This yields two predictions:

(P1) Convergence rates are independent of ν_r .

(P2) Errors grow like $O(\nu_r)$ at fixed mesh.

Nothing in the solver knows what α is. If the coercivity estimate were wrong, in particular if α degraded with ν_r , prediction (P1) would fail.

16.2 Rates across six decades

ν_r	$\mathbf{u}, L^2$	$\mathbf{u}, H^1$	p, L^2	w, L^2	w, H^1
0	3.00	2.00	2.02	3.00	2.00
10^{-2}	3.00	2.00	2.02	3.00	2.00
10^{-1}	3.00	2.00	2.02	3.00	2.00
1	3.00	2.00	2.06	3.05	2.00
10	3.00	2.00	2.77	3.73	2.12
10^2	3.04	2.00	3.56	3.99	2.92
10^3	3.72	2.00	3.62	3.99	2.98

Prediction (P1) holds. The velocity H^1 column reads 2.00 at every parameter value across six orders of magnitude, which is the observable consequence of the coercivity constant being independent of ν_r .

The L^2 columns exceed their theoretical values at large ν_r . Rates above the interpolation order cannot be asymptotic, so this indicates a preasymptotic regime, and inspecting the full rate sequences confirms it. At $\nu_r = 10^2$ the velocity sequence reads 3.52, 3.36, 3.14, 3.04, descending steadily toward 3. At $\nu_r = 10^3$ the transition has not yet reached $h = 1/128$. The picture is of an error of the form $Ah^3 + B(\nu_r)h^k$ with $k > 3$, where the crossover mesh moves finer as B grows. The H^1 seminorm does not carry the higher order contribution and therefore reads the asymptotic rate immediately at every parameter value.

16.3 Growth of the error

At fixed mesh $h = 1/128$:

ν_r	$\mathbf{u}, L^2$ error	w, L^2 error
0	2.6167×10^{-6}	1.3443×10^{-7}
10^{-2}	2.6167×10^{-6}	1.3443×10^{-7}
10^{-1}	2.6167×10^{-6}	1.3447×10^{-7}
1	2.6168×10^{-6}	1.3601×10^{-7}
10	2.6178×10^{-6}	2.1013×10^{-7}
10^2	2.6428×10^{-6}	1.2727×10^{-6}
10^3	4.0822×10^{-6}	1.2025×10^{-5}

Prediction (P2) is sharp for the microrotation, whose error grows by a factor of 9.4 over the last decade, that is, linearly.

It is loose for the velocity, whose error grows by a factor of 1.56 across six decades against a bound permitting 10^3 . For reference, the predicted ratio M/α grows from 2.1 at $\nu_r = 0$ to 9458 at $\nu_r = 10^3$.

16.4 Interpretation

The estimate of Theorem 11.3 is an upper bound, not a growth rate, so slower growth does not contradict it. What would contradict it is growth faster than linear.

The mechanism for the velocity is structural. The viscous term carries the factor $(\nu + \nu_r)$, and so does the manufactured forcing, since the forcing was constructed by substituting a fixed exact solution into the strong form. That factor therefore largely divides out when the velocity block is inverted. The remaining genuinely ν_r dependent contribution comes from the coupling term $2\nu_r \nabla^\perp w$, and its effect on the velocity is damped by the $1/(\nu + \nu_r)$ arising from the same inversion. The two occurrences of ν_r cancel. The microrotation has no such cancellation, since

its diffusion coefficient is c , which does not scale with ν_r , and there the growth is genuinely linear.

This phenomenon is documented. It is the viscosity induced locking effect familiar from the pressure-robustness literature for the Stokes equations, where precisely this experimental setup, a fixed exact solution with the forcing rescaled by the parameter, is used to demonstrate the difference between pressure-robust and non-pressure-robust methods. The observation is therefore real and correctly diagnosed, but not new.

The estimate as proved is posed on the product space and controls all three fields through a single constant, so it cannot express an asymmetry in which the velocity component is robust and the microrotation component is not. Whether a componentwise estimate capturing this can be established, and whether the pressure-robust construction extends to the micropolar system where two parameters enter four terms rather than one, is not addressed here.

17 Summary

Established	Method
Poisson discretisation correct	MMS, rates 3 and 2
Stokes discretisation correct	MMS, rates 3, 2, 2
Micropolar discretisation correct	MMS, rates 3, 2, 2, 3, 2
Coercivity constant independent of ν_r	(P1) across six decades
Error bound sharp in w	(P2), factor 9.4 per decade
Error bound loose in $\mathbf{u}$	factor 1.56 over six decades

The practical lessons from the implementation, in order of how much time they save:

Derive the forcing by symbolic differentiation rather than by hand. Never interpolate the exact solution when computing an error, and raise the quadrature degree. Assert convergence rates in tests rather than plotting them, and print the whole rate sequence on failure. Prefer tests that assert mathematical properties over tests that inspect degree of freedom numbering. Wrap physical parameters as constants so that forms survive the limit in which a parameter vanishes. Build in stages, verifying each before adding the next, so that a wrong rate has one candidate cause rather than four.